

Distributed Tic-Tac-Toe - Report

Martin Tomassi

1 Problem analysis

Developing a distributed Tic-Tac-Toe game over Java RMI introduces several concurrency and system design challenges. Because independent clients interact with shared server state through synchronous remote invocations, five core problems must be addressed at an architectural level.

The first issue concerns **concurrency in matchmaking**. Since the server processes incoming remote calls concurrently, global registry operations are vulnerable to race conditions. Without proper mutual exclusion, simultaneous requests targeting the same match identifier can cause inconsistent registry states, duplicate match room allocations or erroneous request rejections.

The second problem concerns **maintaining the consistency of the game state**. The board state, turn sequence, and match lifecycle represent shared mutable data. Concurrent move submissions or a player quitting while a move is in progress can lead to illegal state transitions. The system must also process disconnections and abandonments deterministically, ensuring that final outcomes leave the game in a valid state.

The third concern relates to **client responsiveness**. Since remote calls are synchronous and blocking, executing network calls directly on the client's main UI thread causes the GUI to freeze during network latency and server processing. The client architecture must keep network I/O and state modifications off the Event Dispatch Thread (EDT), while avoiding race conditions between incoming state updates and UI rendering.

The fourth issue concerns the **isolation of server-side callbacks**. If the server sends state update notifications to clients synchronously, a slow, lagging, or unreachable client will block the thread executing the remote action. This delays the response to the active player, propagates network latency to other participants, and stalls overall match progression. The server must isolate the sending of remote updates so that network issues on a single client never delay the server's processing or notifications to the opponent.

The fifth problem concerns **resource lifecycle management on the server**. Active matches allocate server-side resources, and if these are not explicitly released at the end of the game, memory and thread leaks will occur over time.

2 General strategy

To resolve the issue of concurrency in matchmaking, match management is centralized within a single component (`GameManagerImpl`). Match creation, joining, and the replacement of finished games are handled as atomic operations. The manager ensures that competing creation requests targeting the same match identifier resolve deterministically: only one client succeeds in registering the match. Furthermore, terminated matches are atomically removed before the match is assigned.

The consistency of the state for each game is maintained by encapsulating each active match within a dedicated remote object (`GameImpl`) that delegates domain operations to the `GameState` monitor. To protect the shared mutable state, `GameState` enforces mutual exclusion on all operations that modify the state. Upon completing a valid operation, the monitor atomically produces a serializable and immutable record (`GameSnapshot`).

To ensure client-side responsiveness, the client controller (`GameController`) isolates network communication and model state management from the UI. Outbound remote invocations are delegated to a background single-threaded executor (`networkExecutor`), preventing blocking RMI calls from freezing the EDT. For inbound updates, the controller exposes itself as a remote callback object (`PlayerCallback`). Incoming RMI notifications are immediately enqueued onto `networkExecutor` to safely update the local model (`ClientGameModel`). Then, operation completion callbacks and model change listeners dispatch UI updates back onto the EDT via `SwingUtilities.invokeLater`, maintaining strict Swing thread confinement for rendering.

To prevent client-side network latency from blocking the server, outgoing state update notifications, sent to clients, are delegated from RMI threads to a dedicated component (`GameEventNotifier`). When a player performs an action, the server updates the match state, enqueues a delivery task into a per-match single-threaded executor, and immediately returns the RMI response to the caller. In the background, the executor delivers state snapshots (`GameSnapshot`) to both participants sequentially in strict FIFO order. Network transmission errors (`RemoteException`) are caught and logged inside the worker context, guaranteeing that a slow or unreachable client never delays request completion for the active player or blocks updates to the opponent.

Lastly, server-side resource leaks are prevented by directly linking component lifecycles to match state transitions evaluated within `GameImpl`. Reaching a terminal state triggers a deterministic teardown sequence: once game completion is detected, `GameImpl` transfers the final snapshot to `GameEventNotifier` which ensures that the final state update is fully delivered to clients before performing a cleanup operation that unexports the match remote object from the RMI runtime, removes the match from `GameManagerImpl` and terminates the background executor.